



S.B. JAIN INSTITUTE OF TECHNOLOGY MANAGEMENT & RESEARCH, NAGPUR

Practical 04

Aim: Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behavior by creating orphan and zombie processes.

Name: Ayushi Manoj Kukde

USN: EE24TCD001

Semester / Year: 4th

sem/ 2nd year

Academic Session:

2025-2026

Date of Performance:

Date of Submission:

- ❖ **Aim:** Implement process creation using the fork() system call in Linux to generate parent and child processes, retrieve their PID and PPID, and analyse system behaviour by creating orphan and zombie processes.

❖ **Tasks to be done in this Practical.**

1. Adam is working in an IT company. He has been given a task to reduce the load of a system by killing some of the processes running in the LINUX operating system. Which commands will he use to complete the given task with the help of the following operation?
 - Kill processes by name
 - Kill a process based on the process name
 - Kill a single process at a time with the given process ID
2. Write a program for process creation using C
 - Orphan Process
 - Zombie Process
3. Create the process using fork () system call.
 - Child Process creation
 - Parent process creation
 - PPID and PID

❖ **Objectives:**

- a. Implement process creation using the fork() system call to generate parent and child processes.
- b. Retrieve and display the Process ID (PID) and Parent Process ID (PPID) for both processes.
- c. Analyse system behaviour by creating and observing orphan and zombie processes.

❖ **Requirements:**

✓ **Hardware Requirements:**

- Processor: Minimum 1 GHz.
- RAM: 512 MB or higher.
- Storage: 100 MB free space.

✓ Software Requirements:

- Operating System: Linux/Unix-based.
- Compiler: GCC Compiler.
- Text Editor: Nano, Vim, or any preferred editor.

❖ Theory:

In many operating systems, the fork system call is an essential operation. The fork system call allows the creation of a new process. When a process calls the fork(), it duplicates itself, resulting in two processes running at the same time. The new process that is created is called a child process. It is a copy of the parent process. The fork system call is required for process creation and enables many important features such as parallel processing, multitasking, and the creation of complex process hierarchies.

It develops an entirely new process with a distinct execution setting. The new process has its own address space, and memory, and is a perfect duplicate of the caller process.

Basic Terminologies Used in Fork System Call in Operating System

- ❖ **Process:** In an operating system, a process is an instance of a program that is currently running. It is a separate entity with its own memory, resources, CPU, I/O hardware, and files.
- ❖ **Parent Process:** The process that uses the fork system call to start a new child process is referred to as the parent process. It acts as the parent process's beginning point and can go on running after the fork.
- ❖ **Child Process:** The newly generated process as a consequence of the fork system call is referred to as the child process. It has its own distinct process ID (PID), and memory, and is a duplicate of the parent process.
- ❖ **Process ID:** A process ID (PID) is a special identification that the operating system assigns to each process.
- ❖ **Copy-on-Write:** The fork system call makes use of the memory management strategy known as copy-on-write. Until one of them makes changes to the shared memory, it enables the parent and child processes to share the same physical memory. To preserve data integrity, a second copy is then made.
- ❖ **Return Value:** The fork system call's return value gives both the parent and child process information. It assists in handling mistakes during process formation and determining the execution route.

Process Creation

When the fork system call is used, the operating system completely copies the parent process to produce a new child process. The memory, open file descriptors, and other pertinent properties of the parent process are passed down to the child process. The child process, however, has a unique execution route and PID. The copy-on-write method is used by the fork system call to maximize memory use. At first, the physical memory pages used by the parent and child processes are the same. To avoid unintentional changes, a separate copy is made whenever either process alters a shared memory page.

The return value of the fork call can be used by the parent to determine the execution path of the child process. If it returns 0 then it is executing the child process, if it returns -1 then there is some error; and if it returns some positive value, then it is the PID of the child process.

Fork() System call

Advantages of Fork System Call

- **Creating new processes** with the fork system call facilitates the running of several tasks concurrently within an operating system. The system's efficiency and multitasking skills are improved by this concurrency.
- **Code reuse:** The child process inherits an exact duplicate of the parent process, including every code segment, when the fork system call is used. By using existing code, this feature encourages code reuse and streamlines the creation of complicated programmes.
- **Memory Optimisation:** When using the fork system call, the copy-on-write method optimises the use of memory. Initial memory overhead is minimised since parent and child processes share the same physical memory pages. Only when a process changes a shared memory page, improving memory efficiency, does copying take place.
- Process isolation is achieved by giving each process started by the fork system call its own memory area and set of resources. System stability and security are improved because of this isolation, which prevents processes from interfering with one another.

Disadvantages of Fork System Call

- **Memory Overhead:** The fork system call has memory overhead even with the copy-on-write optimisation. The parent process is first copied in its entirety, including all of its memory, which increases memory use.
- **Duplication of Resources:** When a process forks, the child process duplicates all open file descriptors, network connections, and other resources. This duplication may waste resources and perhaps result in inefficiencies.
- **Communication complexity:** The fork system call generates independent processes that can need to coordinate and communicate with one another. To enable data transmission across processes, interprocess communication

methods, such as pipes or shared memory, must be built, which might add complexity.

- **Impact on System Performance:** Forking a process duplicates memory allocation, resource management, and other system tasks. Performance of the system may be affected by this, particularly in situations where processes are often started and stopped.

❖ CODES

1. FORK.c

```
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid == 0) {
        // Child process
        printf("Child Process\n");
        printf("Child PID = %d\n", getpid());
        printf("Parent PID = %d\n", getppid());
    }
    else if (pid > 0) {
        // Parent process
        printf("Parent Process\n");
        printf("Parent PID = %d\n", getpid());
        printf("Child PID = %d\n", pid);
    }
    else {
        printf("Fork failed\n");
    }

    return 0;
}
```

2. Orphan.c

```
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid > 0) {
        printf("Parent process terminated\n");
    } else {
        sleep(5);
        printf("Child process running, Parent is dead\n");
    }

    return 0;
}
```

3. Zombie.c

```
M ~
GNU nano 8.7
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid = fork();

    if (pid > 0) {
        // Parent process
        sleep(30);
        printf("Parent process running\n");
    }
    else if (pid == 0) {
        // Child process
        printf("Child process terminated\n");
    }

    return 0;
}
```

❖ Output

Nano fork.c

```
M ~
Dell@DESKTOP-D7MT61A MSYS ~
$ nano fork_pid.c

Dell@DESKTOP-D7MT61A MSYS ~
$ gcc fork_pid.c -o fork_pid

Dell@DESKTOP-D7MT61A MSYS ~
$ ./fork_pid
Parent Process
Parent PID = 1362
Child Process
Child PID = 1363
Child PID = 1363
Parent PID = 1362

Dell@DESKTOP-D7MT61A MSYS ~
$ |
```

Nano orphan.c

```
De1l@DESKTOP-D7MT61A MSYS ~  
$ gcc orphan.c -o orphan  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ ./orphan  
Parent process terminated  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ Child process running, Parent is dead
```

Nano Zombie.c

```
M ~  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ nano zombie.c  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ gcc zombie.c -o zombie  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ ./zombie  
Child process terminated  
Parent process running  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$
```

Kill a single process at a time with the given process ID

```
M ~  
De1l@DESKTOP-D7MT61A MSYS ~  
$ ps  
  PID  PPID  PGID  WINPID  TTY      UID    STIME COMMAND  
  1819   1784   1819   21044  pts/1    197609 16:42:28 /usr/bin/ps  
  1784   1782   1784   19104  pts/1    197609 16:42:19 /usr/bin/bash  
  1782      1   1782   21972  ?        197609 16:42:19 /usr/bin/minTTY  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ ps -ef  
  UID    PID  PPID  TTY      STIME COMMAND  
  De1l   1784   1782  pts/1    16:42:19 -bash  
  De1l  1820   1784  pts/1    16:42:42 ps -ef  
  De1l   1782      1  ?        16:42:19 /usr/bin/minTTY -i C:\msys64\msys2.exe -o AppLaunchCmd=C:\msys64\msys2.exe -o AppID=MSYS2.Shell.MSYS.9 -o AppName=MSYS2 MSYS Shell --store-taskbar-  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ kill 1784  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ kill 1820  
bash: kill: (1820) - No such process  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$ ps -ef  
  UID    PID  PPID  TTY      STIME COMMAND  
  De1l   1784   1782  pts/1    16:42:19 -bash  
  De1l  1821   1784  pts/1    16:44:20 ps -ef  
  De1l   1782      1  ?        16:42:19 /usr/bin/minTTY -i C:\msys64\msys2.exe -o AppLaunchCmd=C:\msys64\msys2.exe -o AppID=MSYS2.Shell.MSYS.9 -o AppName=MSYS2 MSYS Shell --store-taskbar-  
  
De1l@DESKTOP-D7MT61A MSYS ~  
$
```